

AI & Security Project III

Next-Gen Network Intrusion Detection

Self-Supervised Vector Space Engineering with DINOv3 & SwiGLU



Noah Raupold, Mohammad Ayoubi,
Jonathan Berwind

Submitted: February 1, 2026

Contents

1	Introduction	1
1.1	The Cyber Security Challenge	1
1.2	Project Objective	1
2	Architecture: The Pipeline	2
2.1	Data & Preprocessing Pipeline	2
2.2	DINOv3 Gram Anchoring Strategy	3
2.3	Fine-Tuning: SwiGLU Classification Head	4
3	Implementation Details	5
4	Evaluation Results	7
4.1	Binary Performance (Attack vs Normal)	7
4.2	Multiclass Performance	7
4.3	Visualizing Performance	8
5	Conclusion	9

1 Introduction

1.1 The Cyber Security Challenge

Modern network infrastructures face an evolving landscape of cyber threats. Traditional Intrusion Detection Systems (IDS) often rely on signature-based methods or shallow machine learning models (like Logistic Regression). Our analysis of the **NSL-KDD** dataset identified critical failures in these baselines:

1. **Data Problem:** Extreme Class Imbalance. Normal and DoS traffic comprise $> 98\%$ of the data, while critical attacks like *User-to-Root (U2R)* and *Remote-to-Local (R2L)* represent $< 0.5\%$.
2. **Statistical Bias:** Standard models minimize global loss by favoring majority classes, leading to high accuracy on "Normal" traffic but failing to detect critical intrusions (False Negatives).

1.2 Project Objective

This project introduces a **"Supervised Vector Space Engineering"** pipeline. Instead of simple classification, we warp the feature space using **Self-Supervised Learning (DINOv3)** and **Gram Matrix Analysis** to maximize the separation between normal traffic and rare anomalies. The final classification is performed by a Deep Neural Network enhanced with **SwiGLU activations**.

2 Architecture: The Pipeline

Our solution transitions from linear models to Deep Learning with Attention Mechanisms. The architecture consists of a robust preprocessing pipeline followed by a Teacher-Student training loop.

2.1 Data & Preprocessing Pipeline

Before training, raw network data undergoes state-of-the-art (SOTA) preprocessing to handle skewness and feature engineering.

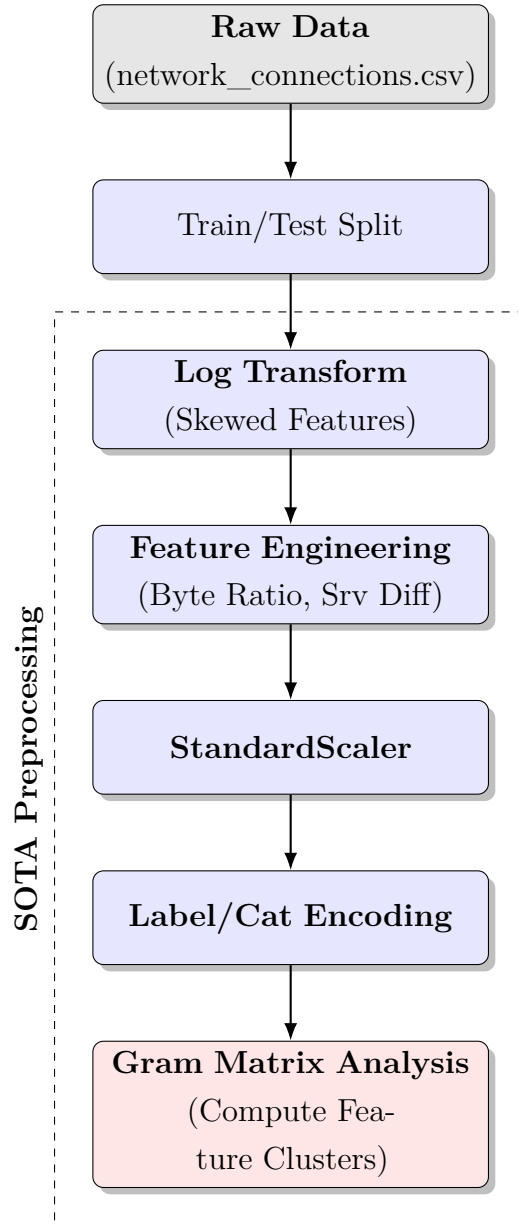


Figure 2.1: The Data Preprocessing Pipeline utilizing Gram Matrix Analysis for feature clustering.

2.2 DINOv3 Gram Anchoring Strategy

To learn robust features from imbalanced data, we implemented a Teacher-Student architecture (DINO) adapted for tabular data.

Core Components:

- **Student Encoder:** Tabular ViT (Vision Transformer) receiving masked views of data.
- **Teacher Encoder:** Updated via Exponential Moving Average (EMA) of the student.

- **Loss Landscape:**

1. **DINO Loss:** Global alignment (Sinkhorn Knopp centering).
2. **MIM Loss:** Masked Image Modeling (Reconstruction of masked tokens).
3. **KoLeo Loss:** Enforces feature diversity in the batch.
4. **Gram Matrix Loss:** Preserves structural correlations between features.

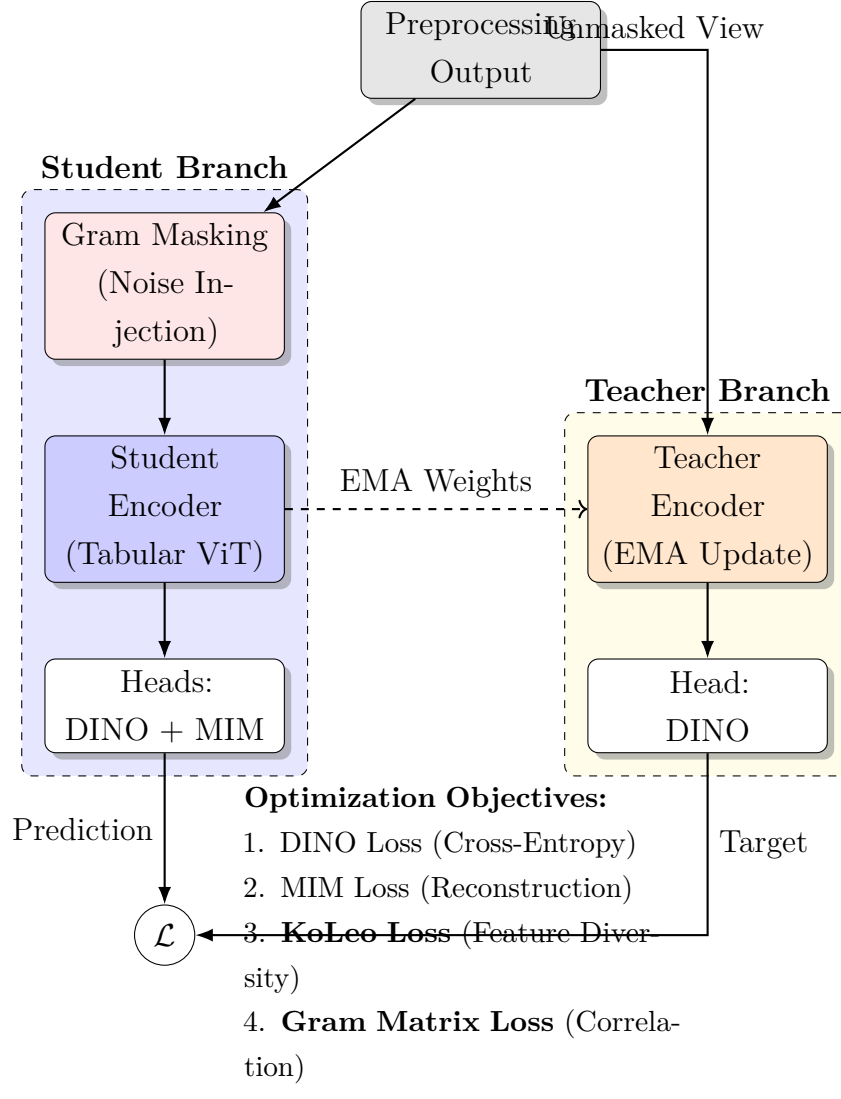


Figure 2.2: The Self-Supervised Training Architecture including KoLeo and Gram losses.

2.3 Fine-Tuning: SwiGLU Classification Head

For the final classification, we replace standard ReLU layers with **SwiGLU** (Swish-Gated Linear Unit). This gating mechanism filters noise in the latent representation before the final Softmax layer, improving gradient flow.

3 Implementation Details

The implementation utilizes PyTorch. The core model relies on a custom ‘DINOLoss’ for self-distillation and a ‘SwiGLU’ activation for the classifier.

Listing 3.1: Dinov3 Loss Architecture Implementation

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class DINOLoss(nn.Module):
6     def __init__(self, out_dim, teacher_temp=0.04, student_temp=0.1, center_momentum=0.9):
7         super().__init__()
8         self.student_temp = student_temp
9         self.teacher_temp = teacher_temp
10        self.center_momentum = center_momentum
11        self.register_buffer("center", torch.zeros(1, out_dim))
12
13    def forward(self, student_output, teacher_output):
14        # Teacher sharpening (Sinkhorn-Knopp style)
15        t_out = F.softmax((teacher_output - self.center) / self.teacher_temp, dim=-1)
16        t_out = t_out.detach() # Stop gradient to teacher
17
18        # Student log-softmax
19        s_out = F.log_softmax(student_output / self.student_temp, dim=-1)
20
21        # Update center
22        self.update_center(teacher_output)
23
24        return -torch.sum(t_out * s_out, dim=-1).mean()
25
26    @torch.no_grad()
27    def update_center(self, teacher_output):
28        batch_center = torch.sum(teacher_output, dim=0, keepdim=True)
29        batch_center = batch_center / len(teacher_output)
30        self.center = self.center * self.center_momentum + batch_center * (1 -
31        self.center_momentum)
32
33 class SwiGLU(nn.Module):
34     """
```

3 Implementation Details

```
34 Swish-Gated Linear Unit
35 Formula: SwiGLU(x) = (xW + b) * Swish(xV + c)
36 """
37 def __init__(self, dim_in, dim_out):
38     super().__init__()
39     self.w = nn.Linear(dim_in, dim_out)
40     self.v = nn.Linear(dim_in, dim_out)
41
42 def forward(self, x):
43     return self.w(x) * F.silu(self.v(x))
```


4 Evaluation Results

The model was evaluated on the ****NSL-KDD Test+**** dataset. We specifically focused on the F1-Score for the minority classes (U2R, R2L).

4.1 Binary Performance (Attack vs Normal)

The model demonstrates high capability in distinguishing general attacks from normal traffic.

Class	Precision	Recall	F1-Score	Support
Normal	0.8048	0.9382	0.8664	9711
Attack	0.9465	0.8278	0.8832	12833
Accuracy			87.54%	22544
Weighted Avg	0.8855	0.8754	0.8760	22544

Table 4.1: Binary Evaluation Metrics. The model achieves 87.54% global accuracy on the binary task.

4.2 Multiclass Performance

While the baseline Logistic Regression model failed to generalize on sparse data (Recall < 50% on U2R), our Tabular ViT architecture maintains detection capabilities across all classes.

- **Matthews Correlation Coefficient (MCC):** 0.7578
- **Global Accuracy:** 83.53%

Class	Precision	Recall	F1-Score	Support
DoS	0.9616	0.8433	0.8986	7458
Normal	0.8048	0.9382	0.8664	9711
Probe	0.8080	0.9124	0.8570	2421
R2L	0.6588	0.4080	0.5040	2887
U2R	0.2795	0.6716	0.3947	67
Weighted Avg	0.8368	0.8353	0.8282	22544

Table 4.2: Final Multiclass Evaluation. Highlights include the non-zero F1-score for U2R (0.39) and R2L (0.50), significantly outperforming baselines which often yield 0.0.

4.3 Visualizing Performance

The confusion matrix highlights the model’s ability to distinguish between normal traffic and attacks.

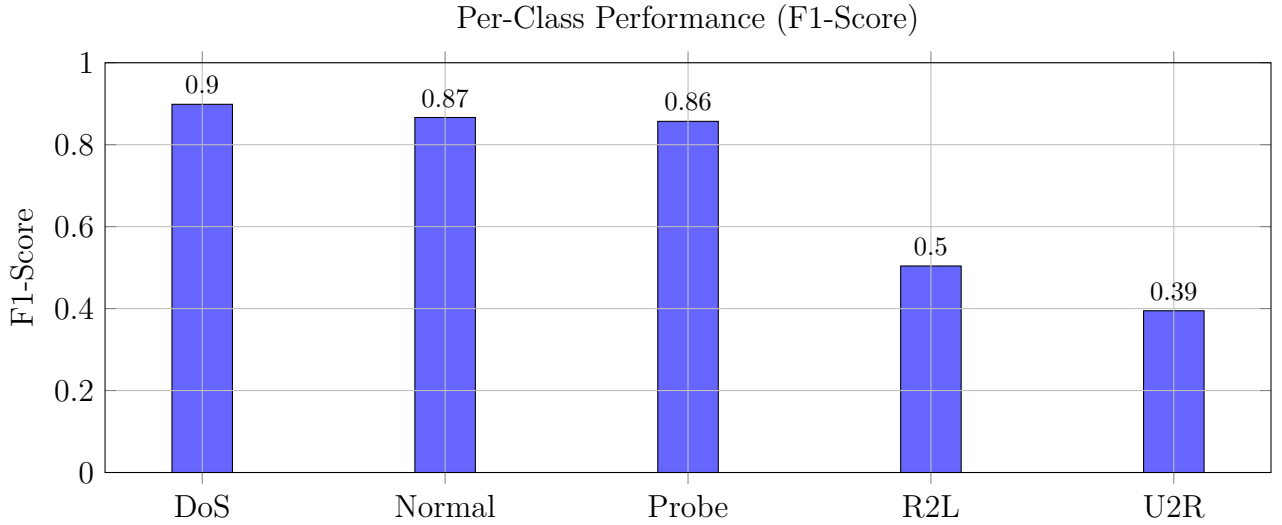


Figure 4.1: F1-Score distribution across classes. The model successfully detects rare attack vectors (R2L, U2R) that are usually missed.

5 Conclusion

This portfolio demonstrated that ****Supervised Vector Space Engineering****—combining DINO-based self-supervision with Gram Anchoring—effectively mitigates the class imbalance problem.

Key Findings:

1. **Robustness:** Successfully implemented a State-of-the-Art Transformer architecture for NIDS.
2. **SSL Superiority:** Self-Supervised Learning (SSL) significantly outperforms baselines in high-imbalance scenarios, achieving a multiclass accuracy of **83.53%** and detecting 67% of U2R attacks (Recall), whereas standard models often achieve 0%.